ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL
FACULTAD DE INGENIERIA EN ELECTRICIDAD Y COMPUTACION

AÑO:	2018	**PERIODO:**	PRIMER TÉRMINO
MATERIA:	SISTEMA DE BASES DE DATOS	**PROFESORES:**	IRENE CHEUNG GINGER SALTOS JOSÉ VILLA JOSÉ CÁRDENAS FRANK MALO
EVALUACIÓN:	SEGUNDA	**FECHA:**	28/08/2018

NOMBRE: __ANULADO_____ MATRICULA: _____

Sección 1 (20%): Conceptos.

a) (10 puntos) Elija una o mas opciones para cada concepto.

a) ACID b) Aislamiento c) Atomicidad d) B+Tree e) Begin f) Clustered g) Commit h) Consistencia i) Delete j) Durabilidad k) End l) Execute m) Flush privileges n) Índices o) Insert p) Integridad q) ORM r) Primary key s) Refresh privileges t) Rollback u) Select v) Set password w) Update x) Unclustered	1. ___**M**____ Actualiza los permisos de un usuario luego de asignar permisos. 2. ___**J**____ Asegura que una transacción perdure, aunque el sistema falle. 3. ___**L**____ Comando que permite asignar permisos para llamar procedimientos almacenados. 4. _**D y F, ó N**_ Estructuras creadas sobre campos claves, a partir de sus datos ordenados, aumentando la velocidad de búsqueda. 5. ___**B**___ Evita que una transacción afecte a otra. 6. ___**R**___ La mayoría de los DBMSs la indexan automáticamente. 7. _**H ó P**_ Protege los datos de la manipulación no intencional o maliciosa. 8. __**Q**___ Son librerías que convierten una relación a objeto para poder ser manipulado en la aplicación. 9. __**T**___ Comando que permite revertir las operaciones de una transacción.

b) (10 puntos) Escoja la opción correcta y justifique su respuesta.

	OPCIONES	**RESPUESTA Y JUSTIFICACION**
1)	A. revoke update from user on db.tbl.col; B. revoke update on db.tb.coll from user; C. revoke update(col) from user on db.tb; **D. revoke update(col) on db.tb from user;**	**Puesto que la columna debe ir entre paréntesis y el usuario al final.**

2)	A. Los disparadores son un conjunto de sentencias que pueden ser ejecutadas por un select, insert, update o delete. B. Los disparadores son un conjunto de sentencias que pueden ser ejecutadas antes o después de un select, insert, update o delete. C. Los disparadores son un conjunto de sentencias que pueden ser ejecutados por el usuario antes o después de un insert, update o delete. **D. Los disparadores son un conjunto de sentencias que pueden ser ejecutados antes o después de un insert, update o delete.**	**Los disparadores no son lanzados por el usuario, ni por una sentencia de tipo *select*.**
3)	**A. ORM disminuye el tiempo de desarrollo de una aplicación.** **B. ORM permite cambiar de motor de base de datos, aunque no es recomendable.** C. ORM es un estándar que lo pueden utilizar en cualquier base de datos. D. ORM mejora el rendimiento de la ejecución de queries.	**- El ORM no es un estándar.** **- La utilización de ORM suele reducir el rendimiento de un query.**
4)	A. Es posible desbloquear una tabla con el comando 'UNLOCK TABLE tabla1' B. No es posible crear un índice sobre dos columnas a la vez. C. Las propiedades de los disparadores son conocidas como ACID. **D. Todos los motores de base de datos incluyen una base de estadísticas que permiten optimizar los queries.**	**- El comando para desbloquear una tabla es UNLOCK TABLES** **- Si es posible crear índices sobre dos o mas columnas.** **- ACID son propiedades de transacciones.**
5)	A. Las vistas no se pueden crear utilizando joins. **B. Las vistas no se actualizan cuando se crean utilizando agrupaciones y campos agregados.** C. Las vistas no permiten compartir información y por ello son seguras. D. Las vistas permiten compartir información y por ello no son seguras.	**- Las vistas pueden ser creadas utilizando joins aunque no se actualicen.** **- Se suelen utilizar para compartir información cambiando los nombres de las columnas por ello, son mas seguras que las tablas.**

Sección 2 (30%): Queries.

Utilice el esquema de base de datos de la Unidad de Forense y escriba los queries necesarios que permitan:

A. (7 puntos) Mostrar el listado de categorías con el número de equipos distintos utilizados en sus procedimientos. Incluya sólo la descripción y el total de equipos para cada categoría.

```
SELECT c.descripcion, COUNT(T.equipo) AS cantidad
FROM categorías c JOIN (
        SELECT DISTINCT categoría, equipo
        FROM Procedimiento JOIN Procedimiento_Equipo
        ON p.idProcedimiento = pe.procedimiento ) T
ON T.categoria = c.idCategoria
GROUP BY c.descripcion;
```

B. (5 puntos) Determinar el origen étnico de la persona más alta en cuyo cuerpo se haya practicado al menos un procedimiento.

```
SELECT oe.descripcion
FROM Origen_Etnico oe JOIN Cuerpo c ON c.origenEtnico = oe.idOrigen
JOIN Cuerpo_Procedimiento cp ON c.idCuerpo = cp.idCuerpo
WHERE c.altura = (
        SELECT MAX(altura) FROM Cuerpo
        JOIN Cuerpo_Procedimiento cp ON c.idCuerpo = cp.idCuerpo) ;
-----------------------------------------------------------------------------------------------------
SELECT oe.descripcion, c.altura
FROM Origen_Etnico oe JOIN Cuerpo c ON c.origenEtnico = oe.idOrigen
JOIN Cuerpo_Procedimiento cp ON c.idCuerpo = cp.idCuerpo
WHERE c.altura > ALL(
        SELECT altura FROM Cuerpo
        JOIN Cuerpo_Procedimiento cp ON c.idCuerpo = cp.idCuerpo) ;
-----------------------------------------------------------------------------------------------------
SELECT oe.descripcion, c.altura
FROM Origen_Etnico oe JOIN Cuerpo c ON c.origenEtnico = oe.idOrigen
JOIN Cuerpo_Procedimiento cp ON c.idCuerpo = cp.idCuerpo
ORDER BY c.altura DESC
LIMIT 1;
```

C. (10 puntos) Encontrar los cinco procedimientos que más se hayan realizado en cuerpos de género femenino de origen étnico 'mestizo' y 'afroamericano'.

```
SELECT p.detalle, Proc FROM
    (SELECT DISTINCT p.detalle, COUNT(p.idProcedimiento) AS Proc
    FROM Cuerpo c JOIN Origen_etnico oe ON c.origenEtnico = oe.idOrigen
    JOIN Cuerpo_Procedimiento cp ON c.idCuerpo = cp.idCuerpo
    JOIN Procedimiento p ON cp.idProcedimiento = p.idProcedimiento
    WHERE c.genero = 'femenino' AND oe.descripcion = 'mestizo'
    GROUP BY p.detalle
    UNION
    SELECT DISTINCT p.detalle, COUNT(p.idProcedimiento) AS Proc
    FROM Cuerpo c JOIN Origen_etnico oe ON c.origenEtnico = oe.idOrigen
    JOIN Cuerpo_Procedimiento cp ON c.idCuerpo = cp.idCuerpo
    JOIN Procedimiento p ON cp.idProcedimiento = p.idProcedimiento
    WHERE c.genero = 'femenino' AND oe.descripcion =  'afroamericano'
    GROUP BY p.detalle) T
ORDER BY Proc DESC
LIMIT 5;
-----------------------------------------------------------------------------------------------------
SELECT DISTINCT p.detalle, COUNT(p.idProcedimiento) AS Proc
FROM Cuerpo c JOIN Origen_etnico oe ON c.origenEtnico = oe.idOrigen
JOIN Cuerpo_Procedimiento cp ON c.idCuerpo = cp.idCuerpo
JOIN Procedimiento p ON cp.idProcedimiento = p.idProcedimiento
WHERE c.genero = 'femenino' AND
            (oe.descripcion = 'mestizo' OR oe.descripcion =  'afroamericano')
GROUP BY p.detalle
ORDER BY Proc DESC
LIMIT 5;
```

D. (8 puntos) Encontrar los empleados que hayan realizado más de 100 procedimientos en cuerpos de origen étnico 'mestizo' en el año 2018. Muestre solamente la cédula y el género del empleado con el total de procedimientos.

```
SELECT cp.idEmpleado, e.genero, COUNT(cp.idProcedimiento) AS Total
FROM Empleado e JOIN Cuerpo_Procedimiento cp ON e.cedula = cp.idEmpleado
JOIN Cuerpo c ON cp.idCuerpo = c.idCuerpo
JOIN Origen_Etnico oe ON c.origenEtnico = oe.idOrigen
WHERE oe.descripcion = 'mestizo' AND c.fechaRecibido LIKE '%2018%'
GROUP BY cp.idEmpleado
HAVING Total > 100;
```

<u>Sección 3 (20%):</u> Queries Avanzados.

Utilice el esquema de base de datos de la Unidad de Forense y escriba las sentencias necesarias que permitan:

A. (12 puntos) Llevar las estadísticas de las Categorías de los procedimientos realizados a los cuerpos por cargo de empleados. Con el afán de no sobrecargar el procesamiento de la base de datos ejecutado una gran consulta bajo demanda, se ha decidido tener una tabla donde se mantendrá este indicador cada vez que se realice un procedimiento a un cuerpo.
La tabla de nombre EstadisticasCatEmp ya existe y tiene la siguiente estructura:

descripcion_categoria	descripcion_cargo	Total

Usted deberá crear una estructura que permita mantener esta tabla, tomando en cuenta las siguientes consideraciones:

- Asuma que la tabla EstadisticasCatEmp hasta el momento del inicio del examen ya se encuentra con los datos iniciales.
- Desde el inicio del examen hasta cualquier momento en el futuro se pueden crear categorías o cargos nuevos en la base de datos.
- Las descripciones son de tipos de datos varchar y el total es de tipo int.

```
DELIMITER $
CREATE TRIGGER tr1 AFTER INSERT ON Cuerpo_Procedimiento
BEGIN
        SET @cargo = SELECT ca.descripcion
        FROM Empleado e JOIN cargos ca ON e.cargo = ca.titulo
        WHERE e.cedula = old.cedula;

        SET @categoria = SELECT ca.descripcion
        FROM Procedimiento p JOIN Categoria ca ON p.categoria = ca.idCategoria
        WHERE p.idProcedimiento = old.idProcedimiento;

        IF @cargo IN (SELECT descripcion_cargo FROM EstadisticasCatEmp) AND
        @categoria IN (SELECT descripcion_categoria FROM EstadisticasCatEmp) THEN
                SET @valor = SELECT Total FROM EstadisticasCatEmp WHERE
                descripcion_cargo = @cargo AND descripcion_categoria = @categoria;
                UPDATE EstadisticasCatEmp SET Total = @valor+1 WHERE
                descripcion_cargo = @cargo AND descripcion_categoria = @categoria;
        ELSE
                INSERT INTO EstadisticasCatEmp VALUES (@categoria, @cargo, 1);
        END IF;
END $
DELIMITER;
```

B. (8 puntos) Tener un reporte el cual permita conocer las estadísticas de las causas de muerte por origen étnico donde el estado se encuentre en 'avanzado estado de descomposición', 'quemado' o 'mutilado'. La estructura del reporte debe ser el siguiente:

causa	etnia	estado	cantidad

```
CREATE VIEW reporte AS
SELECT c.causaDeMuerte AS causa, oe.descripcion AS etnia, e.descripcion AS estado,
COUNT(c.causaDeMuerte) AS cantidad
FROM Origen_Etnico oe JOIN Cuerpo ON c.origenEtnico = oe.idOrigen
JOIN Estado ON c.estado = e.idEstado
WHERE e.estado = 'avanzado estado de descomposición'
OR e.estado = 'quemado' OR e.estado =  'mutilado'
GROUP BY etnia, estado;
```

Sección 4 (30%): Transacción, Índices y Optimización.

a) (10 puntos) Suponga que existen 3 operaciones que se realizan a la tabla : "Cuerpo" de la base de datos db_forense (pág. 4)

Query 1	SELECT fechaEstimadaMuerte FROM Cuerpo WHERE causaDeMuerte = 'suicidios';
Query 2	SELECT idCuerpo FROM Cuerpo WHERE fechaRecibido BETWEEN '2017-08-26' AND '2018-08-26';
Query 3	INSERT INTO Cuerpo(idCuerpo, origenEtnico, estado, fechaRecibido, autopsia_SN, genero, altura, peso, causaDeMuerte, origenDelCuerpo) VALUES (567 ,2, 1, now(), false,'f',1.5, 70, 'arroyado por un carro', 'No hay datos')

Para cada escenario, indique una recomendación de optimización y cómo se debería implementar:

Escenario	Frecuencia de Uso	Cantidad de Tuplas de la tabla Cuerpo	Respuesta
a.	**Query1) 5%** **Query2) 90%** **Query3) 5%**	**10.000**	**Crearía un índice sobre el campo fechaRecibido puesto que ahí se utiliza la mayor cantidad de recursos al ser utilizada un 90% veces sobre una tabla de 10.000 registros.** **CREATE INDEX idx1 ON Cuerpo(fechaRecibido);**
b.	**Query1) 5%** **Query2) 15%** **Query3) 80%**	**10.000**	**En caso de que existiese índices, los eliminaría para recuperar recursos, puesto que no es necesario crear índices.**

b) (5 puntos) Consideren las siguientes transacciones:

	T1;	**T2;**
1	start transaction;	
2	insert into Origen_Etnico values (7, 'Indú');	
3	insert into Origen_Etnico values (8, 'Celta');	start transaction;
4	delete Origen_Etnico where IdOrigen = 5;	insert into Origen_Etnico values (9, 'Masai');
5	commit ;	delete Origen_Etnico where IdOrigen = 5;
6		delete Origen_Etnico where IdOrigen = 6;
7		rollback;
8		select count(*) from Origen_Etnico

Si consideramos que antes de ejecutar las transacciones existían 6 filas/tuplas en la tabla Origen_Etnico con identificaciones (idOrigen) desde 1 hasta 6. ¿Cuál es el valor devuelto por el query de T2?

T1 se ejecuta correctamente y por lo tanto se insertan los registros de índice 7 y 8, y se elimina el registro de índice 5. Al Final de T1 hay 7 registros.
T2 incluye un rollback, por lo que ninguna de sus sentencias se ejecutan. Quedando un total de 7 registros.

c) (15 puntos) Diseñe el plan físico optimizado del siguiente query y escriba el query optimizado final en SQL:

$\Pi_{c.idCuerpo,\ c.genero,\ cp.resultadoProcedimiento,\ e.descripcion}$

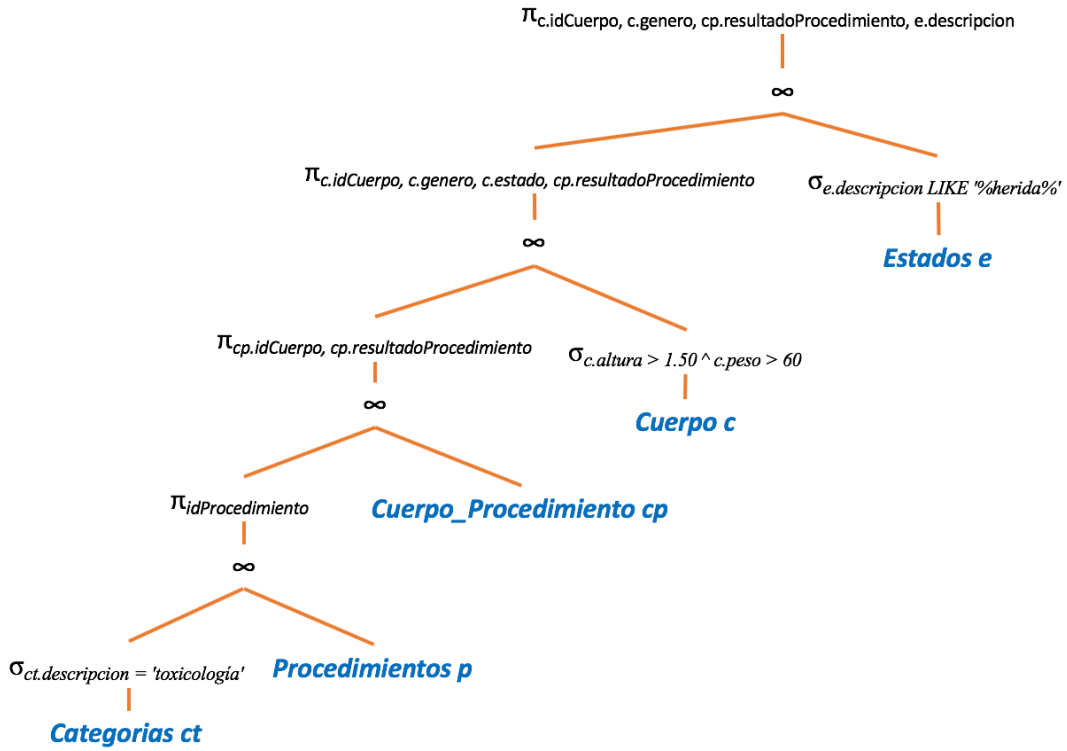
$(\sigma_{c.altura > 1.50\ \wedge\ e.descripcion\ LIKE\ '\%herida\%'\ \wedge\ c.idCuerpo = cp.idCuerpo}$

$(\sigma_{c.peso > 60\ \wedge\ ct.descripcion = 'toxicologia'\ \wedge\ cp.idProcedimiento = p.idProcedimiento}$

$(\sigma_{c.estado = e.idEstado\ \wedge\ p.categoria = ct.idCategoria}$

(Cuerpo c **X** *(Estados e* **X** *(Cuerpo_Procedimiento cp* **X** *(Procedimientos p* **X** *Categorias ct))))))*

8 puntos



7 puntos

```
SELECT Z.idCuerpo, Z.genero, Z.resultadoProcedimiento, e.descripcion
FROM Estado e JOIN
        (SELECT c.idCuerpo, c.genero, c.estado
        FROM Cuerpo c JOIN
                (SELECT cp.idCuerpo, cp.resultadoProcedimiento
                FROM Cuerpo_Procedimiento cp JOIN
                        (SELECT p.idProcedimiento
                        FROM Procedimiento p JOIN Categorias ct
                        ON p.categoria = ct.idCategoria
                        WHERE ct.descripcion = 'toxicología') X
                ON cp.idProcedimiento = X.idProcedimiento) Y
        ON c.idCuerpo = Y.idCuerpo
        WHERE c.altura > 1.50 AND c.peso > 60) Z
ON Z.estado = e.idEstado
WHERE e.descripcion LIKE '%herida%';
```

MODELO LÓGICO NORMALIZADO

